



TECNOLÓGICO
NACIONAL DE MÉXICO®



TECNOLOGICO NACIONAL DE MEXICO CAMPUS LOS MOCHIS

CARRERA:

INFORMATICA

MATERIA:

Administración de proyectos

ASUNTO:

Organización de flujo de trabajo y de desarrollo

INVOLUCRADOS:

Salazar Esquer Cesar

11 de Mayo de 2026

Contenido

1. Entornos de Desarrollo Oficiales.....	3
1.1 Desarrollo Backend	3
1.2 Desarrollo Frontend.....	3
1.3 Base de Datos.....	4
1.4 Testing y APIs	4
2. Organización del Repositorio GitHub.....	5
3. Estructura Oficial de Branches.....	5
3.1 Branch Principal: main	5
3.2 Branch de Desarrollo: develop	5
3.3 Branches de Funcionalidades: feature	6
4. Flujo de Trabajo	6
5. Convención de Commits	7
6. Subida de Cambios.....	8
7. Pull Requests (PR)	8
8. Revisión de Código.....	8
9. Resolución de Conflictos	9
10. Buenas Prácticas Obligatorias	9
11. Estructura del Proyecto.....	10

1. Entornos de Desarrollo Oficiales

Cada integrante deberá utilizar herramientas compatibles con las tecnologías del proyecto para evitar problemas de integración.

1.1 Desarrollo Backend

Herramienta principal:

- IntelliJ IDEA

Uso

El backend será desarrollado en Java utilizando Spring Boot. El programador backend utilizará el IDE para:

- crear controladores
- desarrollar APIs
- implementar lógica de negocio
- gestionar seguridad
- conectar con PostgreSQL

Configuración mínima requerida

- JDK 25
- Maven
- Spring Boot SDK
- Extensiones Git integradas

1.2 Desarrollo Frontend

Herramienta principal:

- Visual Studio

Uso

El frontend trabajará con:

- HTML
- CSS
- Bootstrap
- Razor

El desarrollador frontend será responsable de:

- Interfaces gráficas
- Diseño responsivo
- Formularios
- Integración visual con APIs

Extensiones

- Live Server
- GitLens
- Prettier
- Bootstrap IntelliSense

1.3 Base de Datos

Herramientas oficiales

- PostgreSQL
- pgAdmin

Uso

El DBA o programador SQL será responsable de:

- diseño relacional
- creación de tablas
- relaciones
- consultas
- optimización SQL
- migraciones

1.4 Testing y APIs

Herramienta principal:

- Postman

Uso

El tester verificará:

- endpoints
- respuestas del sistema
- errores
- validaciones
- autenticación

2. Organización del Repositorio GitHub

El repositorio central estará alojado en:

- GitHub

3. Estructura Oficial de Branches

El proyecto utilizará un modelo basado en Git Flow simplificado.

3.1 Branch Principal: main

Función

Contiene únicamente versiones:

- estables
- funcionales
- aprobadas

Reglas

✗ Nadie puede programar directamente aquí.

✗ No se hacen pruebas en esta rama.

✓ Solo el líder técnico puede hacer merge a main.

3.2 Branch de Desarrollo: develop

Función

Es la rama donde se integran todas las nuevas funcionalidades antes de pasar a producción.

Reglas

- ✓ Todos los desarrolladores deben actualizar esta rama antes de comenzar a trabajar.
- ✓ Todos los feature branches salen desde develop.

3.3 Branches de Funcionalidades: feature

Cada integrante trabajará SIEMPRE en su propia rama.

Ejemplos

```
feature/login  
feature/dashboard  
feature/reportes  
feature/usuarios  
feature/tareas  
feature/base-datos
```

4. Flujo de Trabajo

Todos los integrantes deberán seguir exactamente este proceso.

Paso 1: Actualizar develop

Antes de comenzar cualquier trabajo:

- `git checkout develop`
- `git pull origin develop`

Porque garantiza que todos trabajen sobre la versión más reciente.

Paso 2 — Crear rama personal

- `git checkout -b feature/login`

Cada funcionalidad debe tener SU PROPIA rama.

✗ Incorrecto:

`feature-todo`

✓ Correcto:

`feature-login`

`feature-reportes`

`feature-dashboard`

Paso 3 — Programar cambios

Cada desarrollador trabajará únicamente en:

- sus archivos
- su módulo
- su responsabilidad

Paso 4 — Realizar commits frecuentes

Frecuencia recomendada

✓ Commits cada:

- 30 minutos
- cada cambio funcional importante

Ya que de esa forma:

- reduce pérdida de trabajo
- facilita rollback
- mejora trazabilidad
- evita commits gigantes

5. Convención de Commits

Todos los commits deberán seguir una estructura estándar.

Formato

`tipo: descripción corta`

Tipo	Uso
feat	Nueva funcionalidad
fix	Corrección de errores
refactor	Mejora interna
docs	Documentación
style	Cambios visuales
test	Pruebas
sql	Cambios de base de datos

6. Subida de Cambios

Después de hacer commits:

- `git push origin feature/login`

7. Pull Requests (PR)

Cuando una funcionalidad esté terminada:

- Subir rama.
- Crear Pull Request.
- Solicitar revisión.
- Esperar aprobación.

8. Revisión de Código

Antes de aprobar un PR se deberá verificar:

- código limpio
- nombres claros

- funcionamiento correcto
- ausencia de errores
- compatibilidad con otras áreas

9. Resolución de Conflictos

Si dos desarrolladores modifican el mismo archivo, Git generará un conflicto.

Procedimiento correcto

1. Actualizar develop

- `git checkout develop`
- `git pull origin develop`

2. Regresar a la rama

- `git checkout feature/login`

3. Fusionar develop

- `git merge develop`

4. Resolver conflictos manualmente

✗ Nunca borrar código sin revisar.

✓ Comparar cuidadosamente.

10. Buenas Prácticas Obligatorias

Organización

- Mantener carpetas organizadas.
- Separar frontend, backend y SQL.

Código

- Usar nombres descriptivos.

- Comentar lógica compleja.
- Eliminar código innecesario.

Seguridad

- Nunca subir:
- contraseñas
- credenciales
- tokens
- archivos .env
- bases de datos

Git

- Hacer pull antes de comenzar.
- Hacer push regularmente.
- Revisar conflictos antes del merge.

11. Estructura del Proyecto

```
project/  
├── backend/  
├── frontend/  
├── database/  
├── docs/  
├── tests/  
└── README.md
```